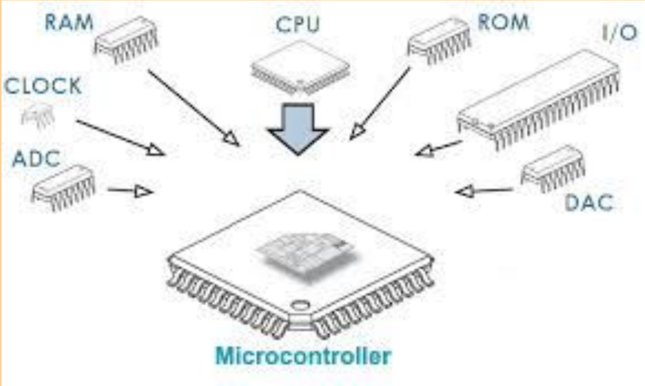
Microcontrollers Workshop

With Arduino Nano

For absolute beginners to learn the basics of Arduino and circuits,
Through a hands on workshop making a few simple projects.

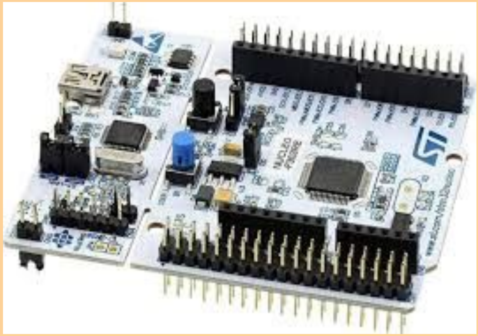
Code and resources found in this github:
TrentChip38/ArduinoActivities

What is a Microcontroller?



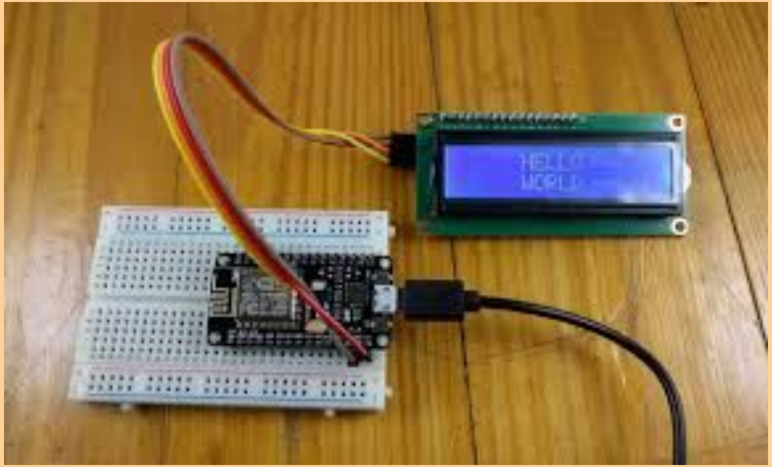
Programmable device

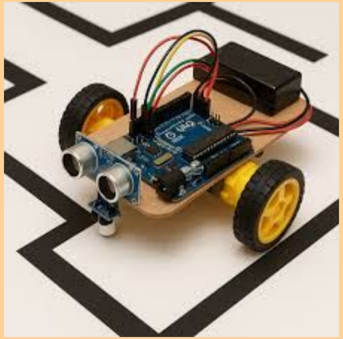
Can read input pins, send on output pins, and communicate with sensors or control motors

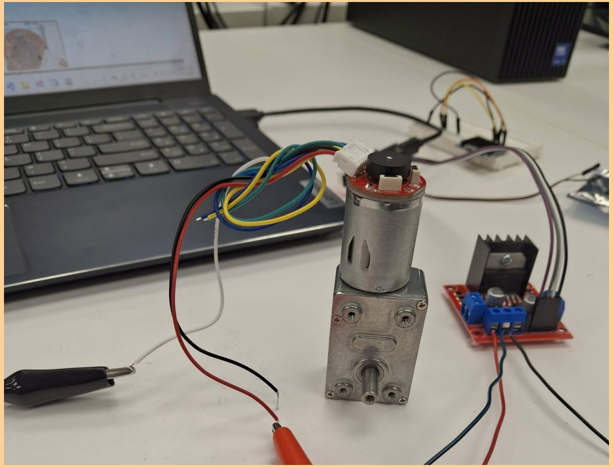


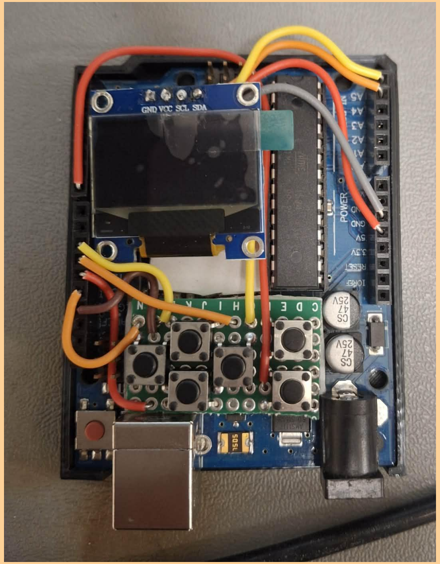
What they can do

Robotics, electronics, LCDs, motor controllers, sensors, and much more.

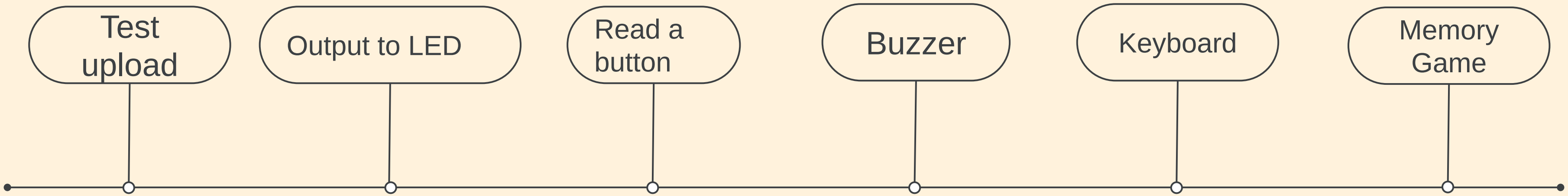








What we will do



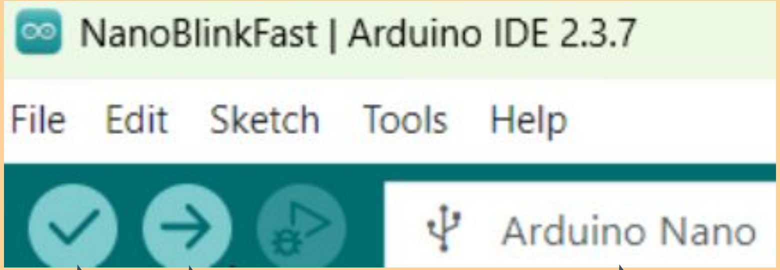
1. Log into the computer

2. Download Arduino IDE 2
    a. Hit windows
    b. Search for “Arduino IDE 2”
    c. Download

3. Open and start a new sketch
    a. Save as

4. Plug in a board
5. Set the com port and type
    a. Type is “Arduino Nano”

Test Upload: Blink

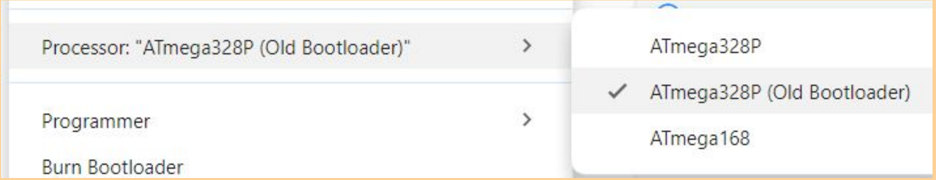


Compile & Upload

Select board

Most likely need to change to the old bootloader:

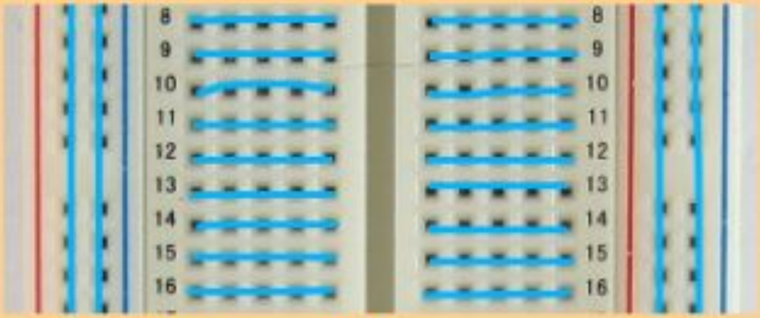
Tools->Processor->Atmega328(Old Bootloader)




```
//Global variable to use
const int mDelay = 200;

//Setup runs once
void setup() {
  //Initialize output pin
  pinMode(LED_BUILTIN, OUTPUT);
}

//Loop runs over and over
void loop() {
  //Turn the LED on (HIGH voltage)
  digitalWrite(LED_BUILTIN, HIGH);
  delay(mDelay); //Wait for 200 milis
  //Turn the LED off with voltage LOW
  digitalWrite(LED_BUILTIN, LOW);
  delay(mDelay); //Wait for 200 milis
}
```

Breadboards

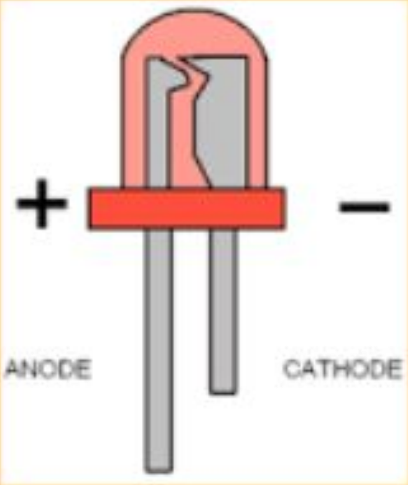
Are used for easy connections



Horizontal rows are connected together on either side and power rails on the edges are connected vertically

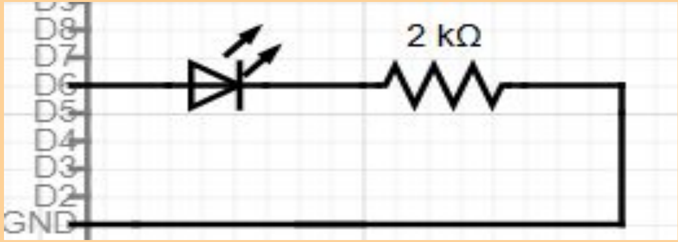
Parts

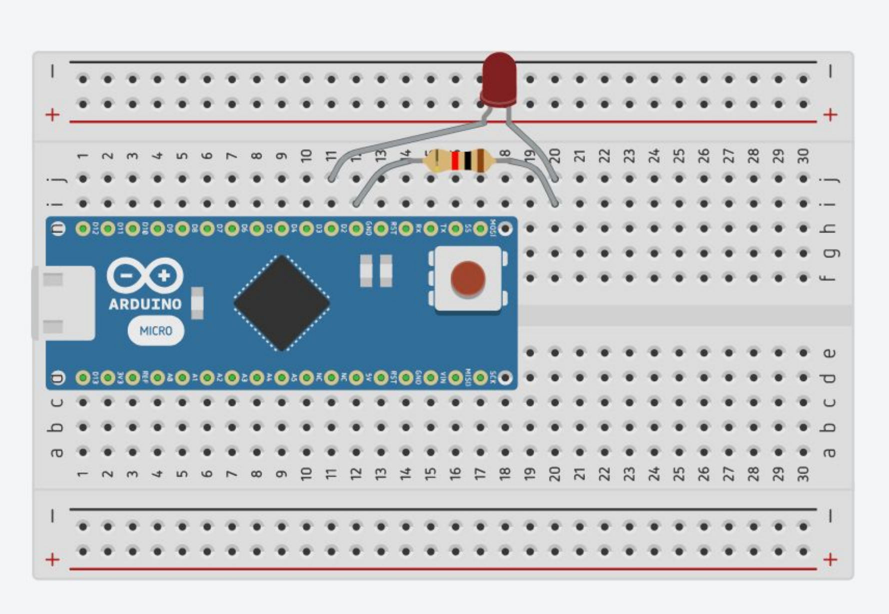
LEDs should have positive connected to their Anode (long end)



The current through an LED must be limited by a resistor so it doesn't blow!

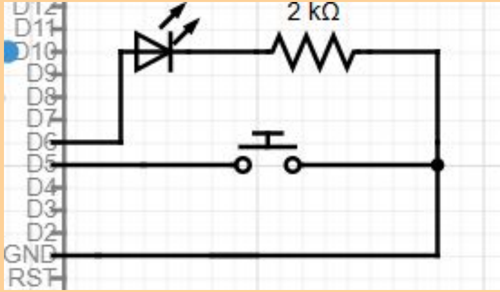
LED Output

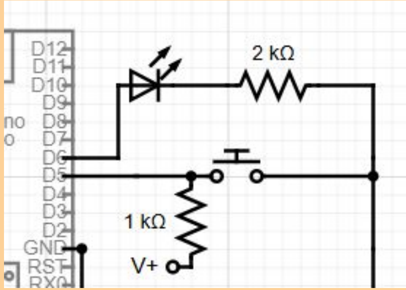


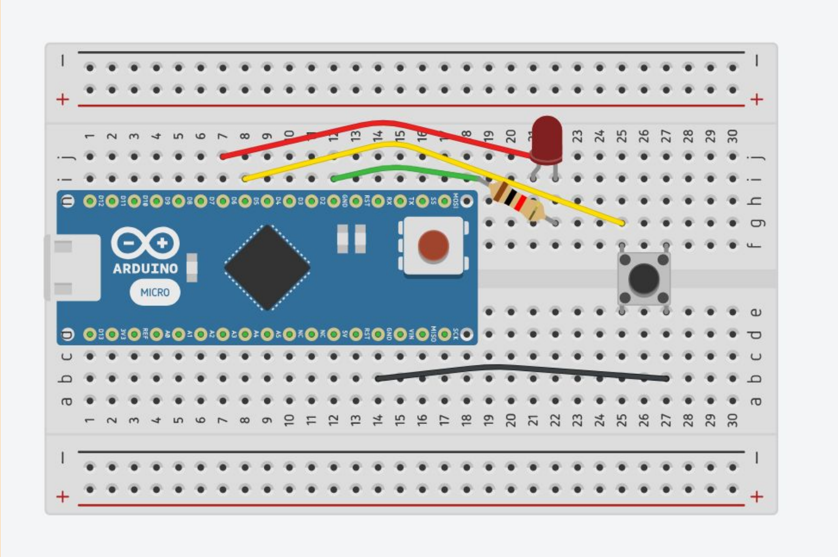


```
const int mDelay = 500;
const int ledPin = 2;

//Setup function runs once
void setup() {
  //Initialize output pin
  pinMode(ledPin, OUTPUT);
}
//Loop function runs over and over
after setup
void loop() {
  //Turn the LED on (HIGH voltage)
  digitalWrite(ledPin, HIGH);
  //Wait for set delay time
  delay(mDelay);
  //Turn the LED off with voltage LOW
  digitalWrite(ledPin, LOW);
  //Wait for set delay time
  delay(mDelay);
}
```

Button Input

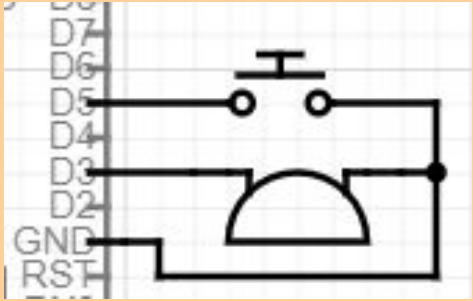


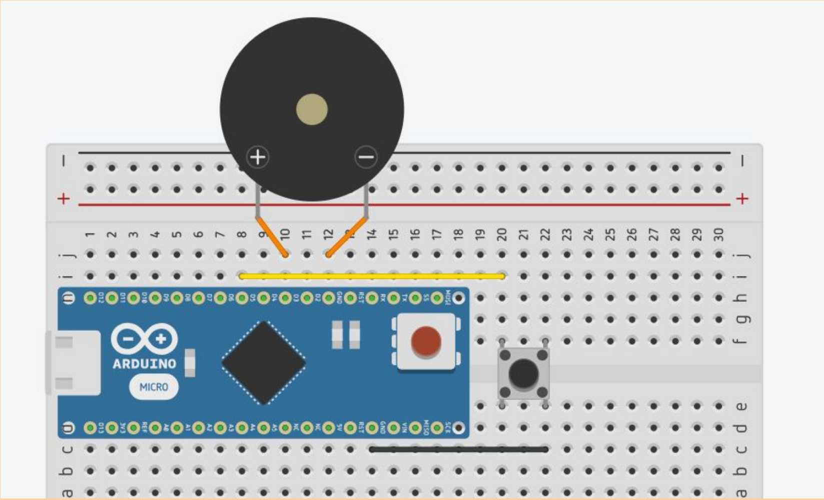





```
//Definitions are similar to variables
#define LEDPin 6
#define btnPin 5

void setup() {
  //Initialize output pin
  pinMode(LEDPin, OUTPUT);
  pinMode(btnPin, INPUT_PULLUP);
}
//Track on/off, start off
int LEDState = HIGH;
void loop() {
  delay(200);//Check button every fith sec
  int buttonState = digitalRead(btnPin);
  //Statement executes if condition is true
  if (buttonState == LOW){
    //Switch LEDState from HIGH to LOW and vv
    LEDState = ~LEDState;
    digitalWrite(LEDPin, LEDState);
  }
}
```

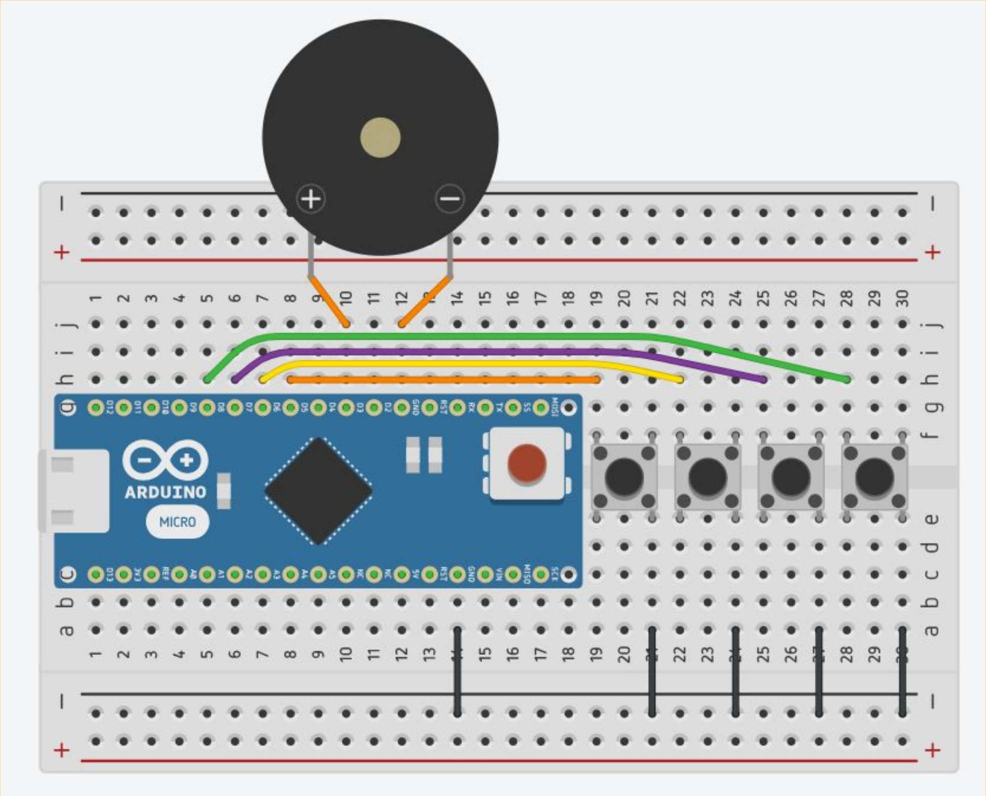
Button & Buzzer





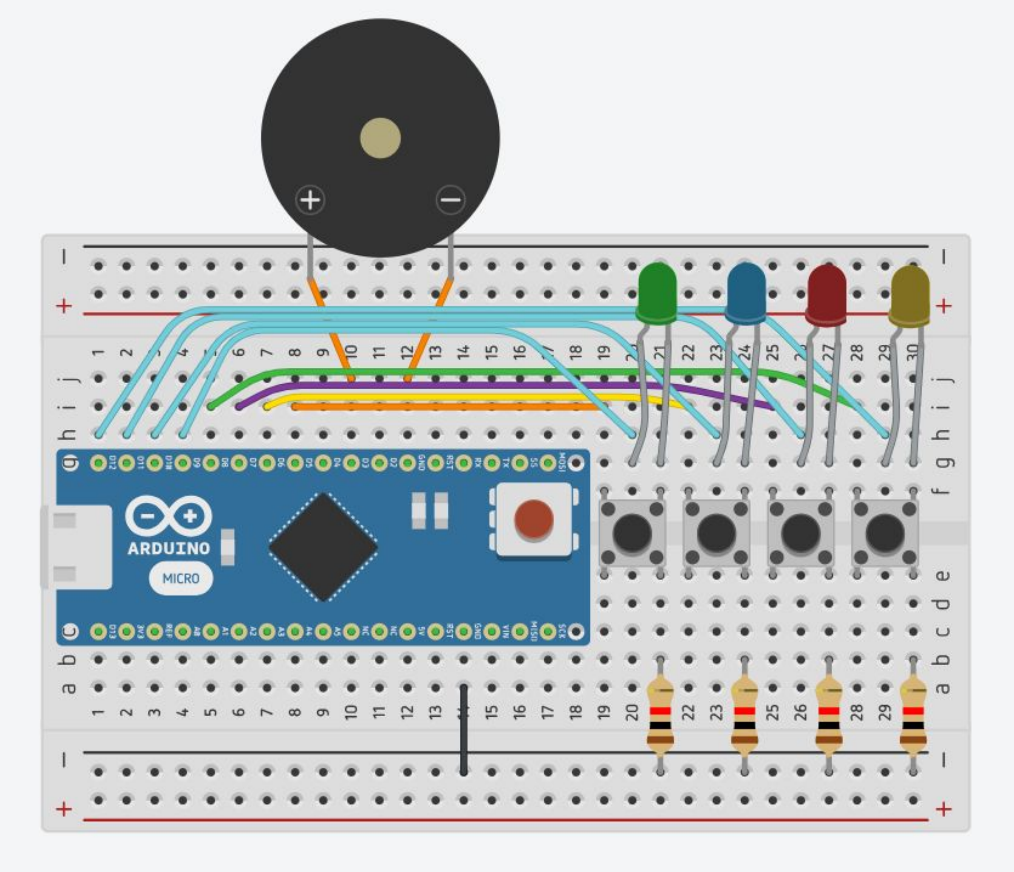

```
//Pin definitions
#define btnPin 5
#define buzzerPin 3

//Setup function runs once
void setup() {
  //Initialize pins
  pinMode(buzzerPin, OUTPUT);
  pinMode(btnPin, INPUT_PULLUP);
}

//Loop function runs over and over
void loop() {
  delay(10);
  if (digitalRead(btnPin) == LOW){
    //Make tone at 440Hz, when pressed
    tone(buzzerPin, 440, 20);
  }
}
```

Keyboard




```
#include "pitches.h"
// Notes to play from the "pitches" library
int notes[] = { NOTE_C4, NOTE_D4, NOTE_E4, NOTE_F4};
const int buttonPins[] = { 5, 6, 7, 8 };
const int buttonCount = 4;
const int speakerPin = 3;
void setup() {
  //Using a for loop to set all the pins
  for (int i = 0; i < buttonCount; i++) {
    pinMode(buttonPins[i], INPUT_PULLUP);
  }
}
void loop() {
  bool buttonPressed = false;
  for (int i = 0; i < buttonCount; i++) {
    if (digitalRead(buttonPins[i]) == LOW) {
      tone(speakerPin, notes[i]);
      buttonPressed = true;
      break; // Only play one tone at a time
    }
  }
  if (!buttonPressed) {
    noTone(speakerPin);
  }
}
```

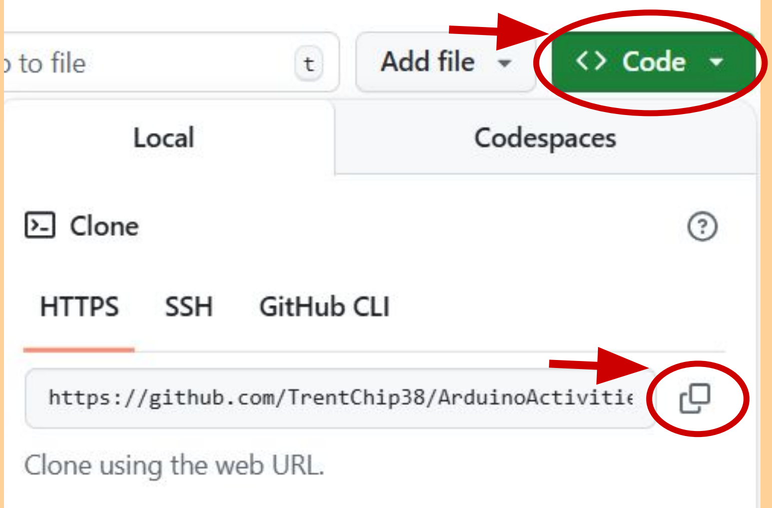
Memory Game



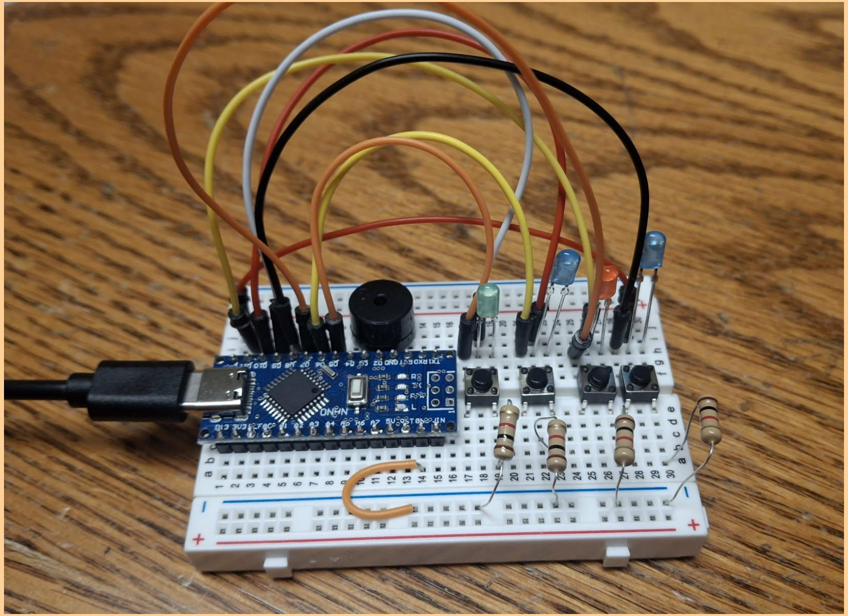
Go to this github:
https://github.com/TrentChip38/ArduinoActivities

And download the zip folder or “git clone” the repo

Then open the final project folder to use the code



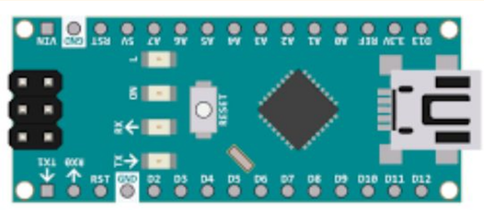
Memory Game



1. Put in the buzzer and buttons first.
   a. Buzzer positive to D3 and ground to GND
   b. Buttons across the bridge however you can fit them.
2. Use the bottom ground rail for negative to the buttons.
3. Then plug in 8 pins from D12-D5. Half are LEDs and other half are buttons.
4. LED above each button with the long positive end connected in between the button legs and gnd to the button ground

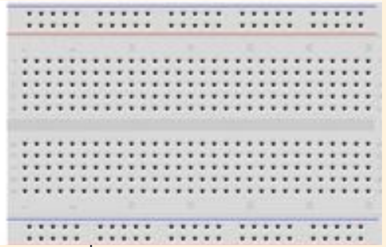
5. Program it and press the right button

Parts we are using



Arduino Nano
Microcontroller

LEDs



Half-size Breadboard

Buttons

Buzzer

1k ohm resistors

8+ cables